

BIG DATA INFRASTRUCTURE & ARCHITECTURE

Big data infrastructure

What kind of hardware do we need?

Scaling

Big data is... *big*

- It doesn't fit a single drive
- It doesn't fit a single (typical) machine



Year: 1990
Size: 1.3 GB
Speed: 4,4 MB/s

5 minutes



Year: 2014
Size: 1 TB
Speed: 100 MB/s

3 hours

(time to read the whole disk)



Year: 2015
Size: 1 TB
Speed: 600 MB/s

30 minutes



Year: 2025
Size: 4 TB
Speed: 6 GB/s

11 minutes

Scaling

Two ways to scale

- **Scale-up** (vertical scaling)
 - Adding more resources (processors, RAM, disks)
 - Upgrading machines (e.g., buying a more expensive and robust server, buying specialized hardware)
- **Scale-out** (horizontal scaling)
 - Adding more machines, creating **clusters**

Shared-memory architecture

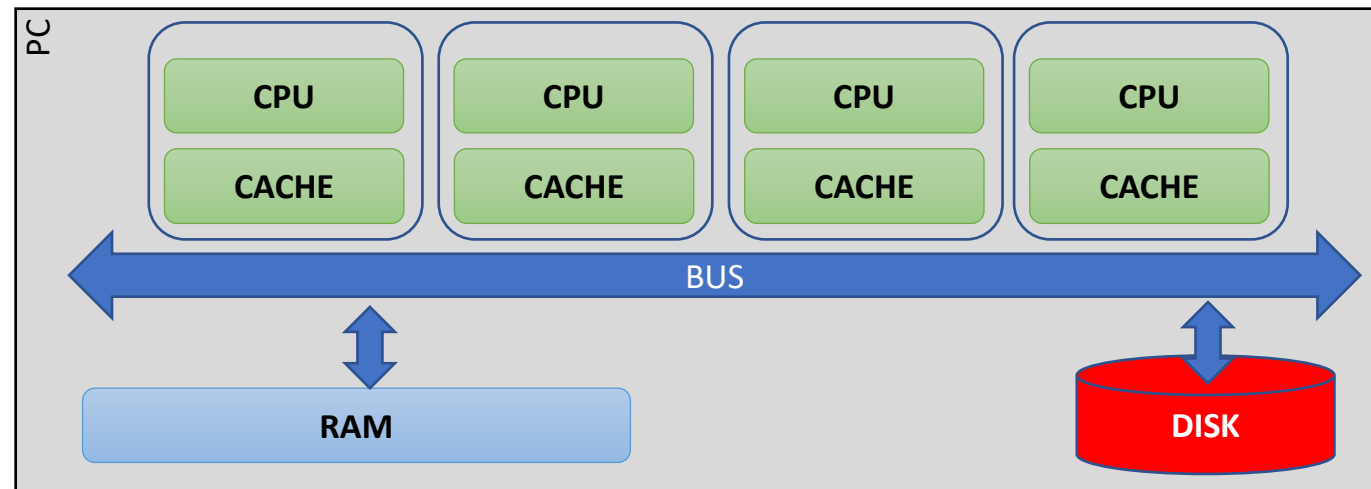
Symmetric MultiProcessing

- Adopted by traditional notebooks and workstations
- Several processors share the same RAM, the same I/O bus and the same disk(s)



Limits

- Physical number of devices that can be mounted or plugged-in
- BUS bottleneck (cannot go over a certain speed)



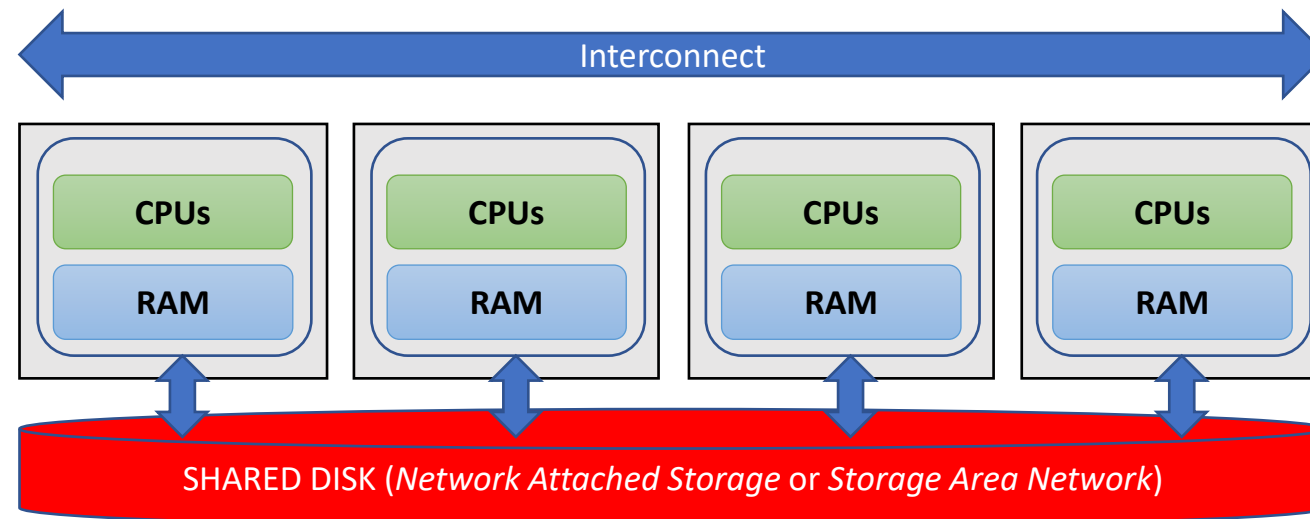
Shared-disk architecture

A group of computing nodes, working together to solve a common goal

- Each (group of) computing unit has its own memory
- (Groups of) computing units communicate with the others through the interconnect

Limits

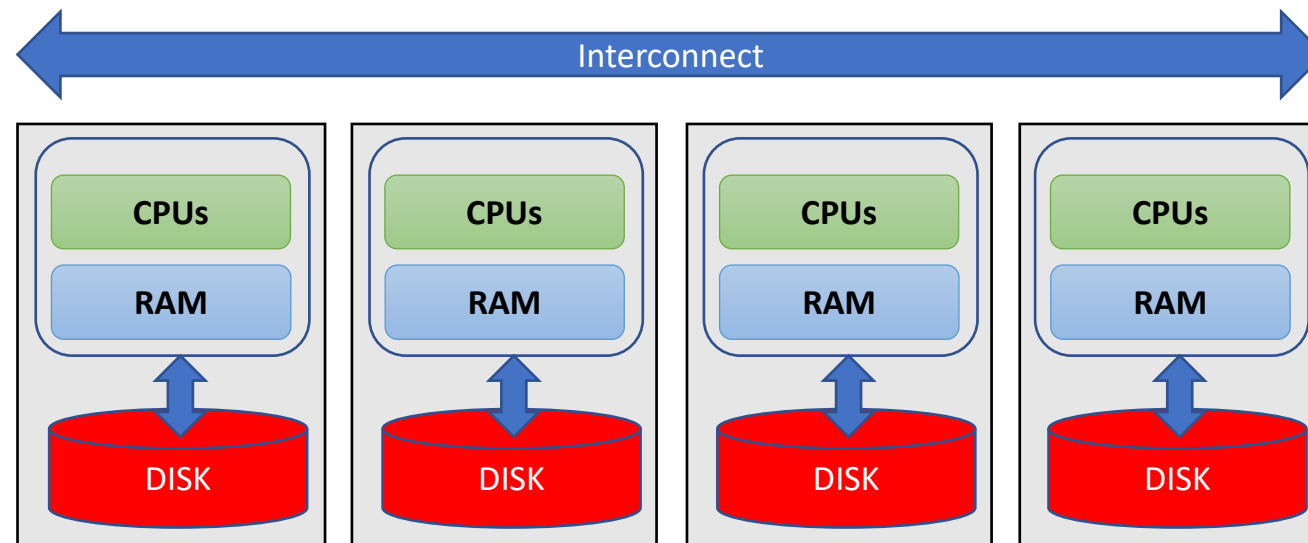
- Not designed for scaling to big data sizes (in terms of internal protocols and hardware)
- Concurrency capabilities are limited



Shared-nothing architecture

A group of computing nodes, working together to solve a common goal

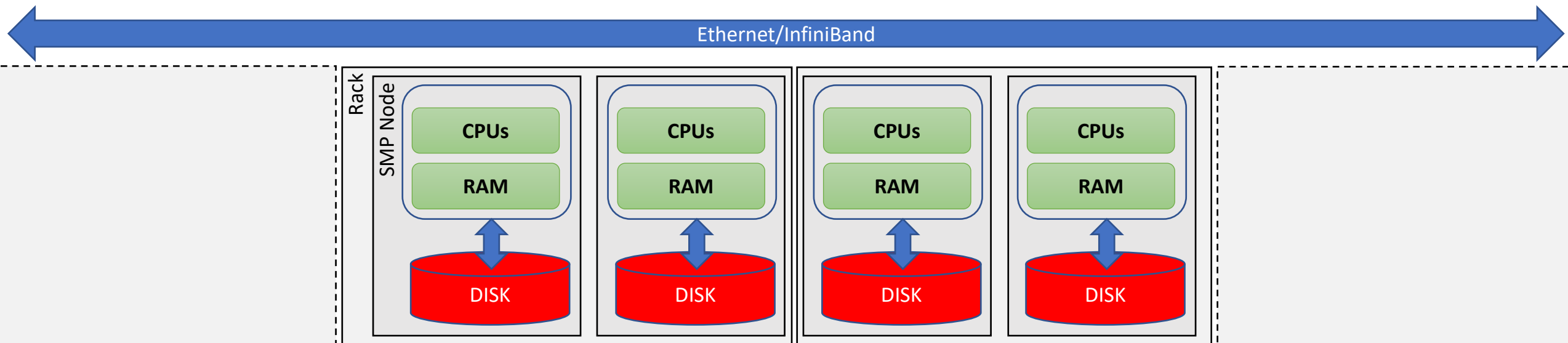
- Each (group of) computing unit has its own resources
- (Groups of) computing units communicate with the others through the interconnect
- Scalability is virtually-unlimited



Cluster

A cluster is a group of autonomous computers (*nodes*) that work together so that they can be viewed as a single system

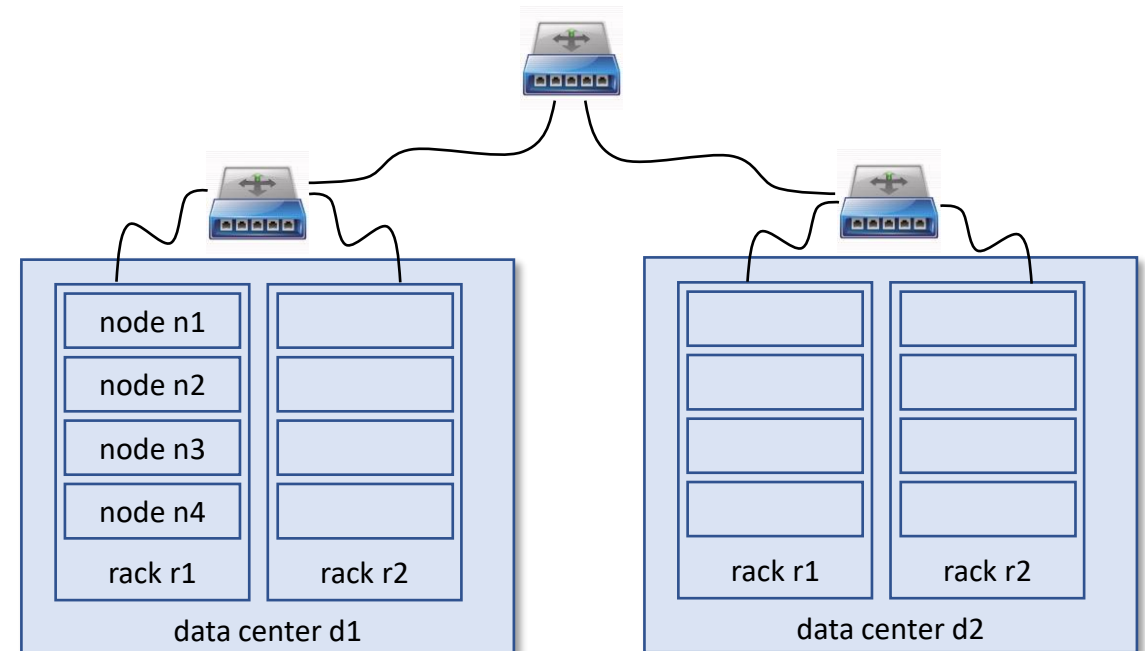
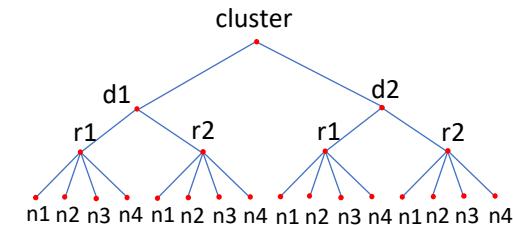
- Every node is a system on its own (SMP)
- Typically connected to each other through fast LANs (Ethernet or InfiniBand)
- Typically adopts a **shared-nothing** architecture
- Stored in racks → Scalability comes at the price of network speed



Cluster

Compute nodes are stored on racks

- 8–64 compute nodes on a rack
- There can be many racks of compute nodes
- **Intra-rack bandwidth $\gg$ inter-rack bandwidth**
- Increasing networking layers decreases the overall bandwidth



Cluster

Clusters can be built/acquired in multiple ways (1/2)

- **Engineered appliance**
 - A specific vendor (e.g., Teradata, IBM) sells/rents a specialized infrastructure
 - Teradata IntelliFlex/VantageCore
 - IBM Netezza Performance Server
 - Microsoft APS/PDW
 - Oracle Exadata
 - Very high performances and no setup issue **BUT** suffers vendor lock-in and limited scalability
- **On-premise**
 - Buy *commodity hardware* and build your own cluster
 - Maximum flexibility **BUT** suffers high setup and maintenance costs, logistic issues, inefficiency risks

Cluster

Clusters can be built/acquired in multiple ways (2/2)

- **IaaS (Infrastructure as a Service)**
 - Rent single machines from a cloud provider
 - Removes logistic costs and adds elasticity **BUT** keeps configuration costs, renting is costly, and there are still inefficiency risks
- **PaaS (Platform as a Service)**
 - Rent managed clusters from a cloud provider
 - Removes configuration costs **BUT** adds an extra layer of costs
- **SaaS (Software as a Service)**
 - Use products/services offered by a cloud provider
 - The cluster is there – you just don't see it

Commodity hardware

Clusters are usually made with **commodity hardware**

- Except with engineered appliances

A *commodity* is something (a good, a product) that

- Can be bought from a wide range of vendors..
- ..at more or less the same price..
- ..without being able of distinguishing where it comes from

Some examples: gold, electricity

Pros

- No vendor lock-in
- Interchangeability/interconnectivity

Cons

- You need to deal with failures
- Commodity $\neq$ low-end! Cheap components with high failure rate can be a false economy

Cluster

Having a single large cluster is tantalizing to many organizations

- No data silos, simpler governance

Multiple clusters are inevitable within medium-large enterprise settings

- Resiliency
 - Every cluster sits within a single point of failure due to geography
- Software development
 - Mitigate the risk of impacting critical production environments by isolating configuration, integration, or evolution testing and deployment
- Workload isolation
 - Hardware resources tuned for specific workloads, less resource contention
- Legal separation
- **Separate storage from compute to scale differently – especially in cloud implementations**

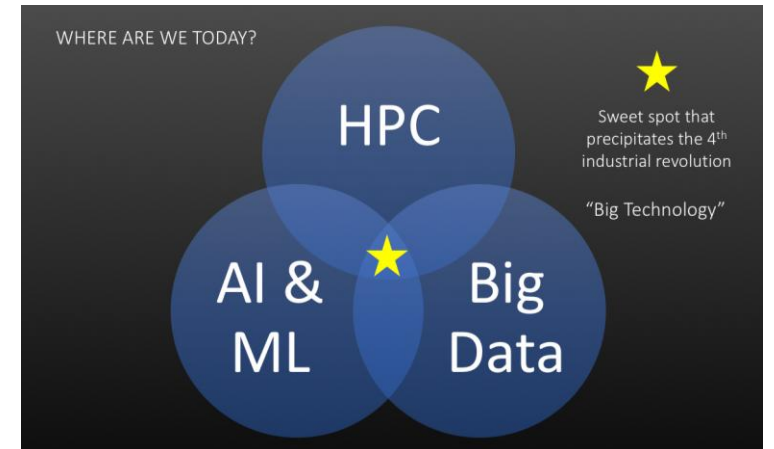
Cluster

What about High-Performance Computing?

- HPC generally refers to massively parallel system specifically devoted to solve **computationally-intensive tasks**
 - HPC **main applications** are **scientific simulations**, **weather forecasts**, **3D modeling**, etc.
 - Usage of GPUs is increasing
- Big Data systems are mostly **data-intensive**
- The line separating Big Data and HPC clusters is very much blurred

Examples @PoliTo

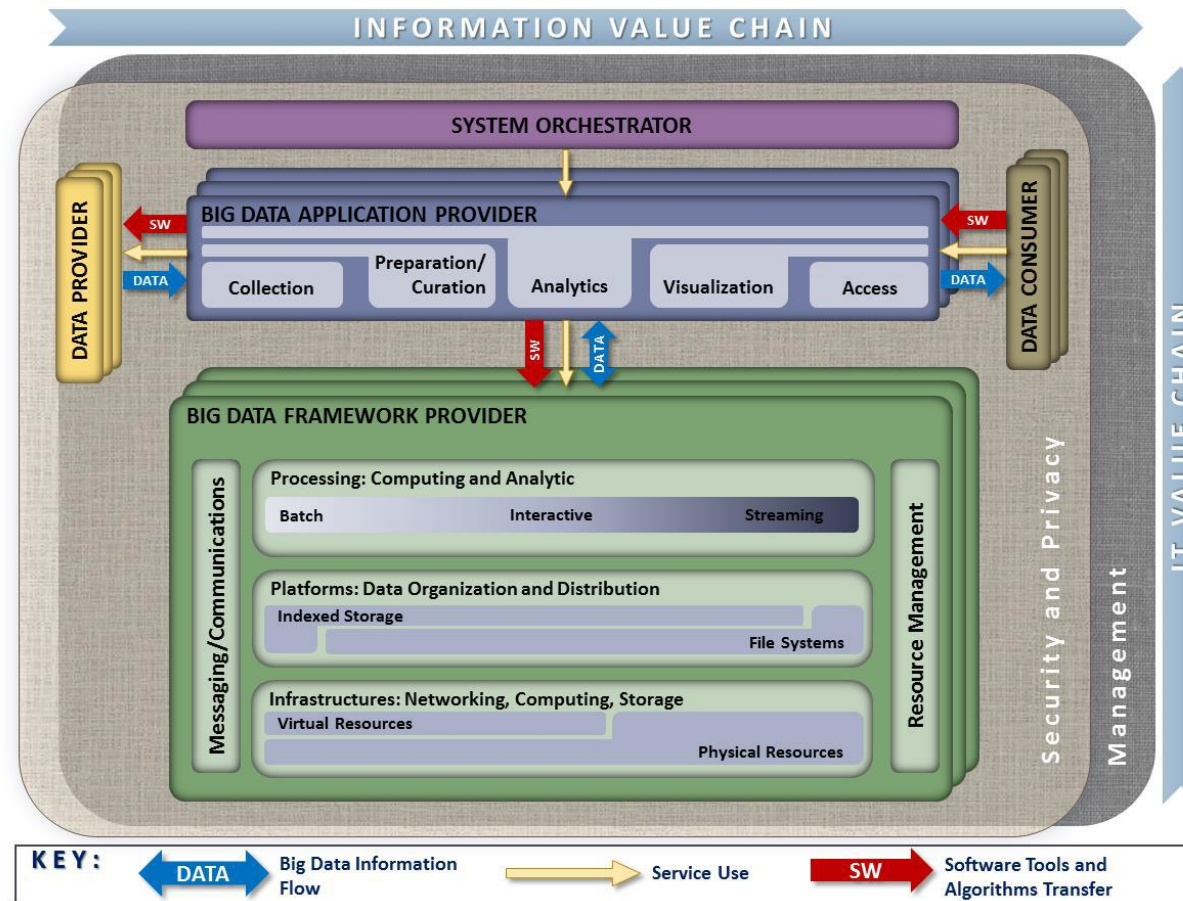
- Big Data computing cluster
 - 1400+ physical cores, 15 TB RAM, 8 PB storage, 6 nVidia V100 GPUs
- HPC computing cluster
 - 2900+ physical cores, 27 TB RAM, 256 TB storage, 32 nVidia V100 GPUs, 100Gb/s Infiniband EDR



Reference Big Data Architectures

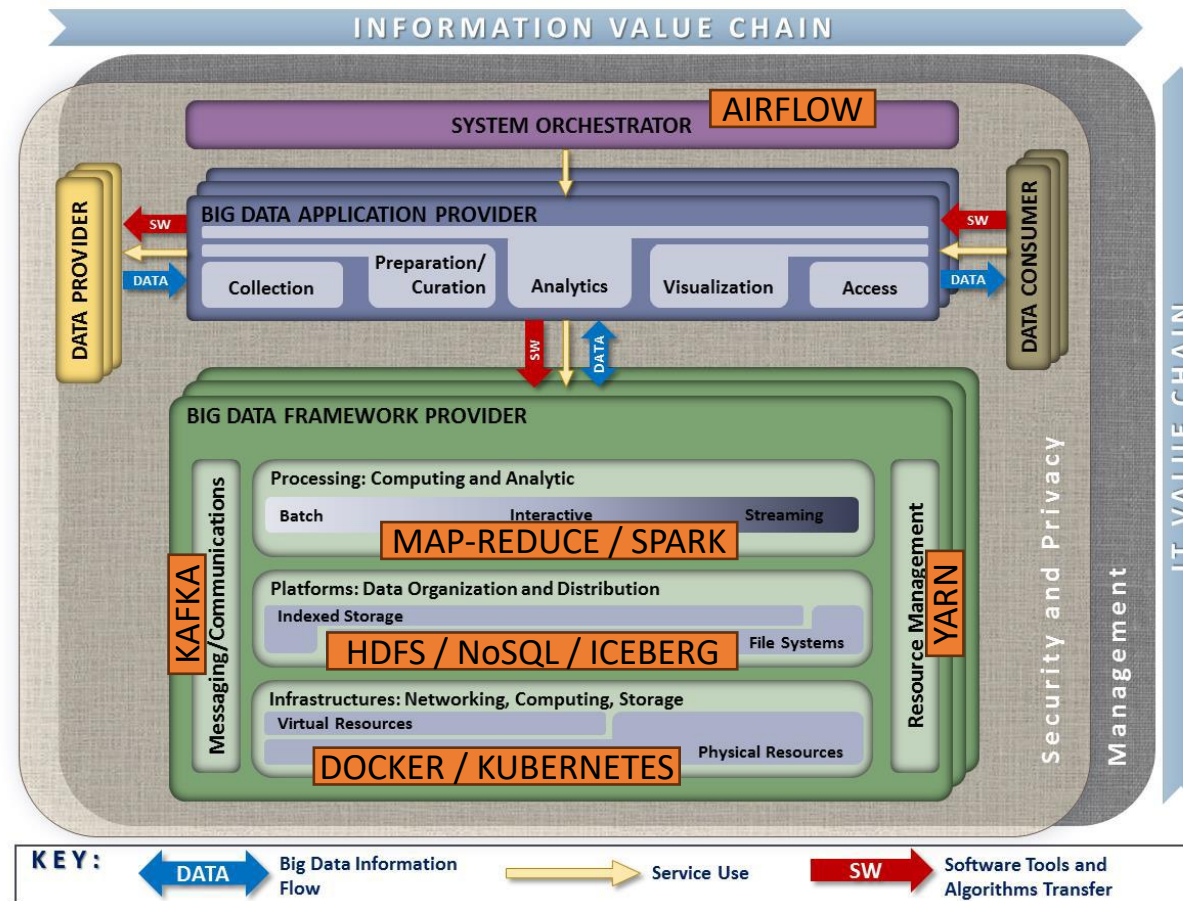
What kind of software do we need?

NIST's reference architecture



<https://bigdatawg.nist.gov/uploadfiles/NIST.SP.1500-6r1.pdf>

NIST's reference architecture



<https://bigdatawg.nist.gov/uploadfiles/NIST.SP.1500-6r1.pdf>

NIST's reference architecture

In short

- **Big Data Framework Provider**
 - The ecosystem of tools (+ the hardware) that you need
 - Provisioning can range from manual (on bare metal) to fully-managed (on cloud)
 - You need to know how the tools work to make your own stack
- **Big Data Application Provider**
 - The programs that must be written to process the data, from collection to visualization
- **System Orchestrator**
 - The puppet master
 - Decides which programs should be run, when, and how
 - Could be manually done or automated

NIST's reference architecture

Big Data Framework Provider

- **Provides general resources or services to be used by the Big Data Application Provider in the creation of the specific application**
 - Handles the communication with the Big Data Application Provider and the assignment of physical resources to the respective activities
- Consists of Infrastructure frameworks..
 - Support the underlying [computing, storage, and networking](#) functions required to implement the overall system
 - Associated with physical or virtual infrastructure resources
- .. Data Platform frameworks..
 - Manage the [organization and distribution of data](#) (file System, databases, etc.)
- .. and Processing frameworks
 - How data will be processed in support of Big Data applications (e.g., [batch/streaming framework](#))

NIST's reference architecture

Data Provider

- **Introduces new data or information feeds into the Big Data system**
- It can be anything from a sensor to a human or another Big Data system
- Includes the following activities:
 - Collecting and persisting the data
 - Providing transformation functions for scrubbing sensitive information
 - Creating the metadata describing the data source, usage policies/access rights, etc.
 - Enforcing access rights on data access and establishing (in)formal contracts for data access authorizations
 - Making the data accessible through suitable programmable push or pull interfaces and mechanisms
 - Publishing the availability of the information and the means to access it

Data Consumer

- **Uses the interfaces or services provided by the Big Data Application Provider to get access to the information of interest**
- It can be an actual end user or another Big Data system

NIST's reference architecture

Big Data Application Provider

- **Executes a specific set of operations (pipeline/workflow)**
 - Meet the requirements established by the System Orchestrator
 - Meet security and privacy requirements
- Includes the following activities:
 - Collection, Preparation, Analytics, Visualization, Access
- Each activity
 - Is specific to the application
 - Can be implemented by independent stakeholders and deployed as a stand-alone service
- There may be multiple and differing instances of each activity, or a single program may perform multiple activities
- Each of the functions can run on a separate Big Data Framework Provider or all can use a common Big Data Framework Provider

NIST's reference architecture

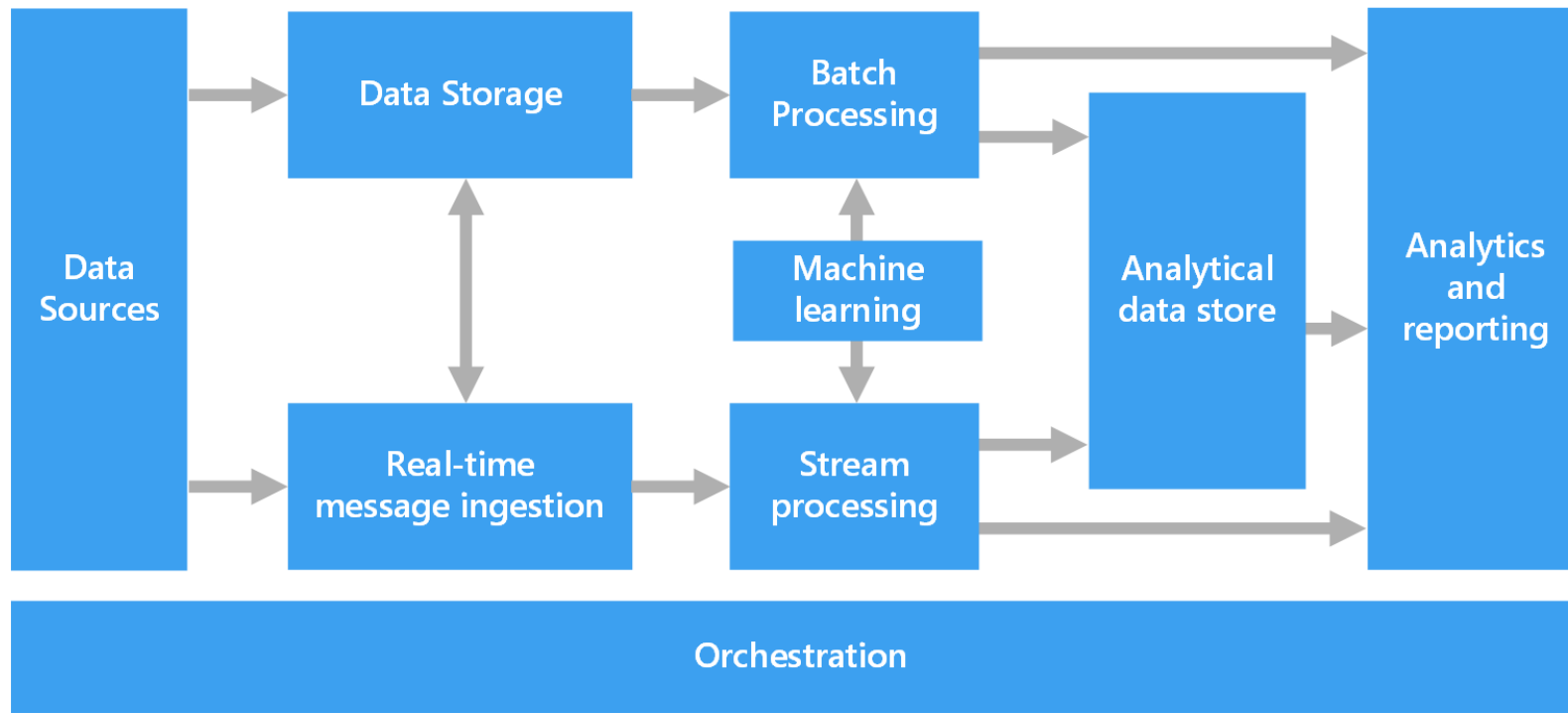
System Orchestrator

- **Integrate the required application activities into an operational vertical system**
- Configure and manage the other components of the Big Data architecture to implement one or more workloads
- Monitor workloads and system to verify the meeting of quality requirements
- Elastically assign and provision additional physical or virtual resources
- **Performed by human and/or software components**

Components of a big data architecture

Logical components that fit into a big data architecture

- Individual solutions may not contain every item



<https://docs.microsoft.com/en-us/azure/architecture/data-guide/big-data/>

Components of a big data architecture

Data sources

- Data stores, static files, real-time sources

Data storage

- Distributed file store (data lake) or database

Batch processing

- Long-running batch jobs to filter, aggregate, and prepare the data for analysis

Analytical data store

- Serve the processed data in a structured format that can be queried using analytical tools

Analysis and reporting

- Traditional OLAP and BI tools, interactive exploration, analytical notebooks

Components of a big data architecture

Real-time message ingestion

- Capture and store real-time messages for stream processing
- Act as a buffer for messages
- Support scale-out processing, reliable delivery, message queueing semantics

Stream processing

- Filter, aggregate, and prepare the data for analysis
- Processed stream data is then written to an output sink

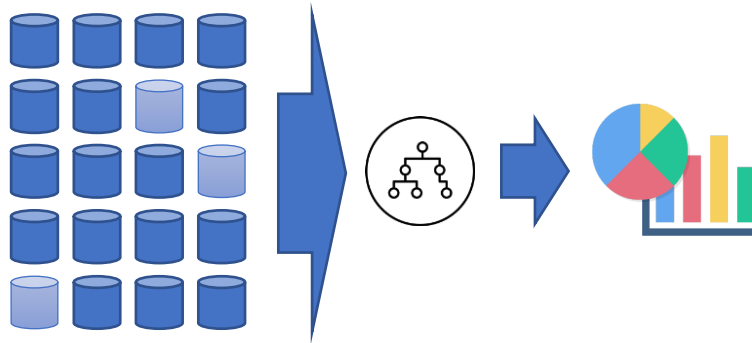
Orchestration

- Automation of repeated data processing operations, encapsulated in workflows
- E.g., transforming source data, moving data between multiple sources and sinks, loading into an analytical data store, pushing results to a dashboard

Two main analytical applications

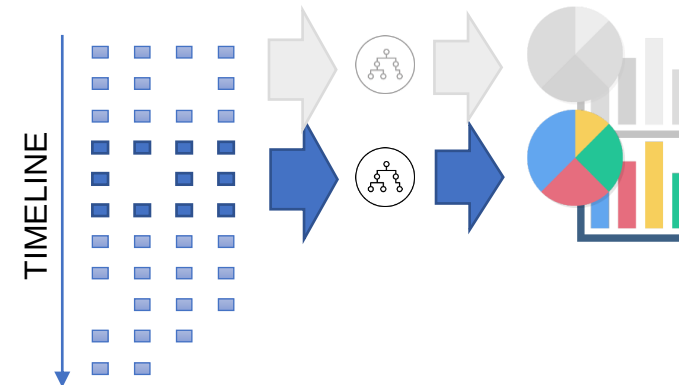
Batch analyses

- Analytical algorithms are launched over large amounts of stored data (at rest)
 - Typical analytical scenario
 - Complex algorithms may take minutes/hours
 - Run whenever needed (*on-demand*)



Stream analysis

- Analytical algorithms are continuously running (non-stop) over potentially infinite data as soon as it is collected
 - Results in near real-time
 - Useful for monitoring purposes
 - One-pass computation

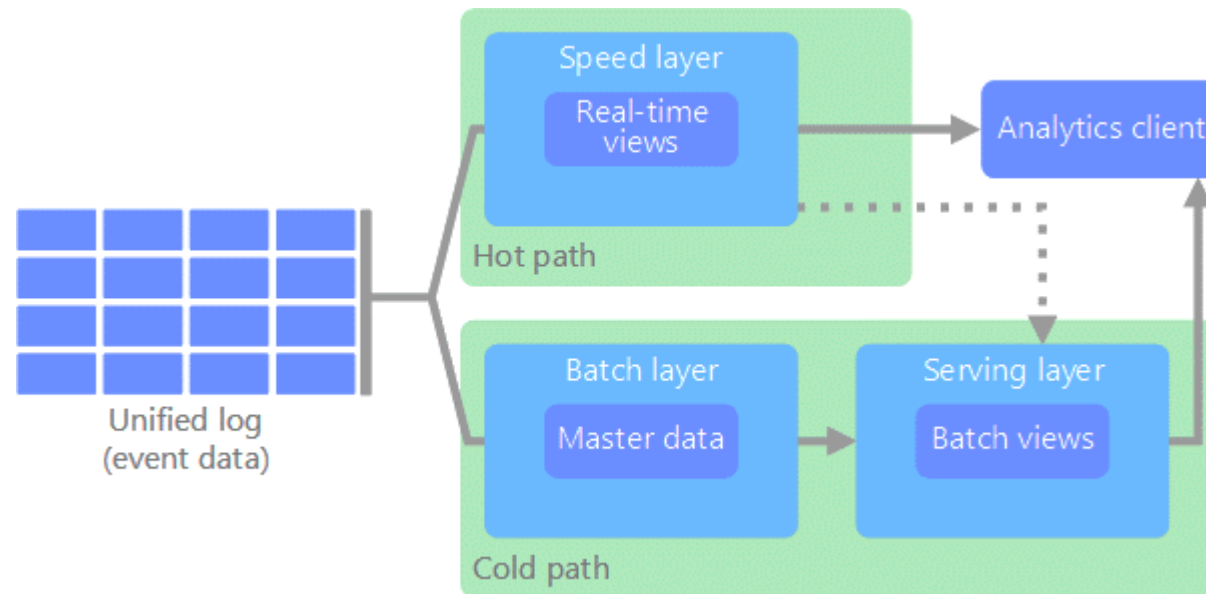


Can we do both on the same data?

Lambda architecture

All data coming into the system goes through two paths

- *Hot path*: for timely, yet potentially less accurate data in real time
- *Cold path*: for less timely but more accurate data

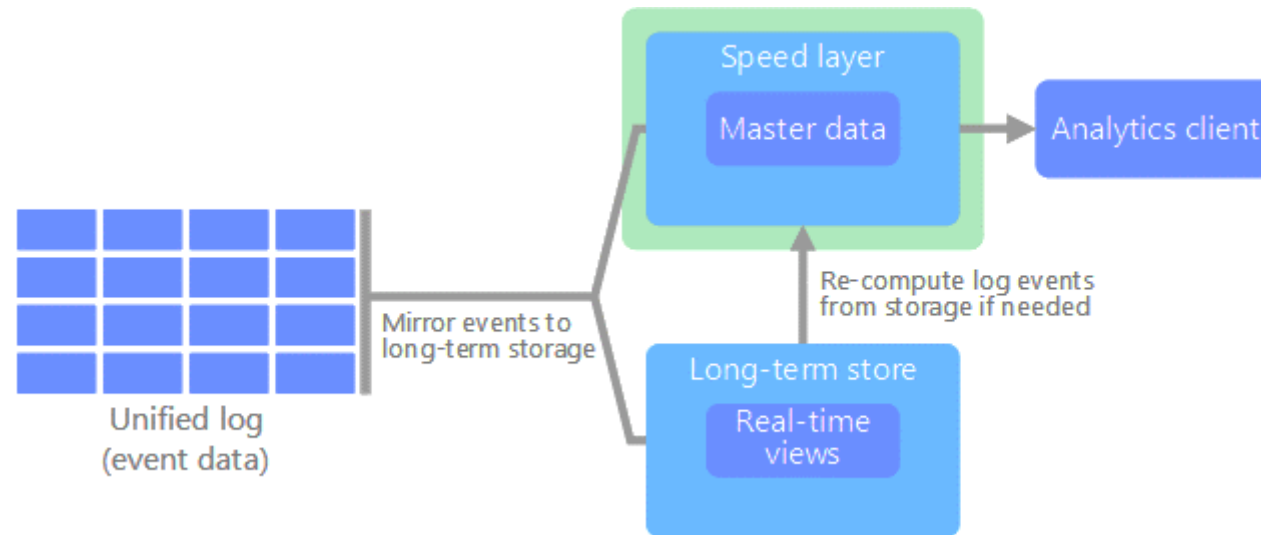


Warren, J. and Marz, N., 2015. *Big Data: Principles and best practices of scalable realtime data systems*. Simon and Schuster.

Kappa architecture

All data flows through a single path, using a stream processing system

- Similar to a lambda architecture's speed layer, all event processing is performed on the input stream and persisted as a real-time view
- To re-compute the entire data set (equivalent to what the batch layer does in lambda), simply replay the stream



Lin, J., 2017. *The lambda and the kappa*. IEEE Internet Computing, 21(5), pp.60-66.

Lambda vs Kappa

Lambda used to be the easiest solution

- But it needs the **parallel development and maintenance of two parallel pipelines**
- Same goal, different languages, different frameworks

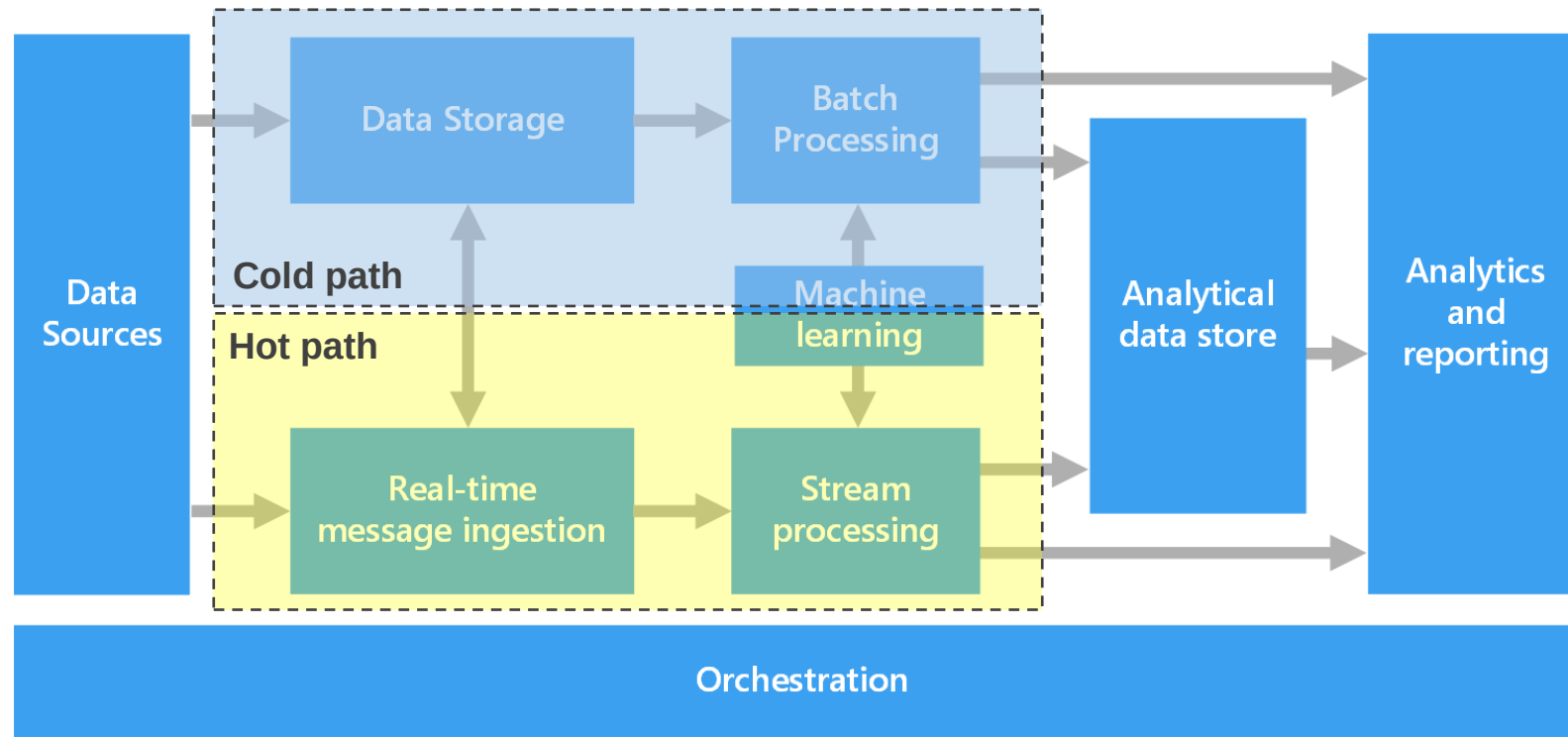
With Kappa, modeling everything under the streaming paradigm is ~~not~~ trivial

- **A streaming engine can handle a bounded dataset**
- Well-designed streaming systems provide a strict superset of batch functionality
 - Introduce correctness (exactly-once semantics, strong consistency) and the result of a streaming job is the same of batch job
- Advanced tools provide a single framework and unified APIs for writing and executing both batch and streaming jobs
 - Google Cloud Dataflow
 - Apache Flink
- *“Broad maturation of streaming systems combined with robust frameworks for unbounded data processing will in time allow for the relegation of the Lambda Architecture to the antiquity of big data history where it belongs”*

Streaming Systems. *The What, Where, When, and How of Large Scale Data Processing*. Tyler Akidau, Slava Chernyak, and Reuven Lax; O'Reilly, 2018

Components of a big data architecture

Overlap with Lambda architecture



Components of a big data architecture

Overlap with Kappa architecture

